Well Written Functions

Writing code that works is only step one. Writing code that you (and your team) can actually read, understand, and debug six months later is what separates junior developers from seniors.

1. The Core of Understandability

An understandable codebase means you can answer two questions almost instantly:

1. **"Where is the feature?"** If the image blurring feature is broken, can you immediately find the exact file and function that handles image blurring?
2. **"Where is the data modified?"** If a user's score is suddenly negative, can you easily track down every single function that has permission to change that score?

2. The Two Causes of a Bug

When a program crashes or produces the wrong result, it always comes down to one of two things:

1. **The logic is wrong:** The function itself is written poorly (e.g., you wrote **x + y** instead of **x * y**).
2. **The data is wrong:** The function is written perfectly, but the data handed to it was already corrupted by a previous step (e.g., **a perfect sorting algorithm was handed an empty list**).

3. The Power of Traceability (And the Danger of Global Variables)

To fix **a "Data is wrong"** bug, you must trace the corrupted data backward to find out which function originally broke it.

- **The Safe Way (Parameters):** If you only pass data into functions via parameters (arguments), tracing is incredibly easy. You just look at the function call, see who called it, and step backward in a straight line.
- **The Dangerous Way (Global Variables):** A global variable is a piece of data stored at the top of your program that any function can access and change at any time without passing it through parameters.
    - **Why it's bad:** If a global variable gets corrupted, tracing the bug is a nightmare. You have to search your entire codebase to find every single function that ever touched that variable, and guess which one broke it.

Expert Takeaway:

In the Go community, there is a famous proverb: **"Clear is better than clever."** When writing complex backend systems, junior engineers often try to write **"clever"** code, chaining 10 operations into a single, unreadable line just to save space.

Senior Go engineers write **"boring",** highly traceable code. They break logic into tiny, well-named functions (e.g., **FetchUser(), ValidateEmail(), SaveToDB())** and pass explicit variables between them. It might take a few extra lines to write, but when the server crashes at 3:00 AM, that traceable, **"boring"** code is exactly what allows you to find and fix the bug in five minutes!